

并行计算 习题一 空白练习卷

来源：习题/ipp-source-main/ipp-source-main/homework/习题一.pdf

说明：本练习卷已删除原 PDF 中填好的答案；选择题、填空题和答题区均留空。

一、填空题

1. 常用的 Linpack 基准测试程序有 _____ 和 _____。
2. MPI 的英文全称为 _____；集合通信的英文为 _____。
3. MPI 的通信分为 _____ 和 _____。
4. 请写出在 3 个主机上启动 MPD 进程，形成 MPI 运行时环境的命令：
_____。（目标主机列表由 \$HOME/mpd.hosts 文件指定）
5. 只有接收进程可以使用通配符参数，常见的通配符参数包括 _____ 和 _____。
6. OpenMP 适用于 _____ 系统进行并行程序设计，其中 #pragma omp critical 的功能是：
_____。
7. OpenMP 中，一条编译指导语句由 _____ 和 _____ 组成。
8. 在 Pthreads 中，引起竞争的代码称为临界区；常用来保护临界区的方式包括：_____
_____ 和 _____。
9. MPI_Scatter(a, local_n, MPI_DOUBLE, local_a, local_n, MPI_DOUBLE, 0, comm) 这条语句执行的功能是：
_____。
10. #pragma omp parallel num_threads(thread_count) reduction(+:global_result) 这条编译指导语句的作用是：
_____。

二、简答题

1. 请简述高性能计算。

答:

2. 在 Linux 系统中，对文件名为 hello.c 的 MPI 源程序、Pthreads 源程序和 OpenMP 源程序，分别写出采用 3 个进程或线程的编译和运行命令。

答:

3. 考虑下面的循环。该循环存在循环依赖，请找一种方法消除循环依赖，并且并行化这个循环。

```
a[0] = 0;
for (i = 1; i < n; i++)
    a[i] = a[i-1] + 2*i;
```

答:

4. 下述代码是非递归深度优先搜索的代码片段，并假设 Feasible(curr_tour, nbr) 始终为真。现考虑 4 个城市的 TSP 问题，请画出栈和当前回路的运行状态，直到 curr_tour 为 _____；请找出下述并行代码的临界区，并采用 OpenMP 的方式解决临界区问题。

```
Push_copy(stack, tour); // 将回路压入栈中
while (!Empty(stack)) {
    curr_tour = Pop(stack); // 将回路弹出栈中
    if (City_count(curr_tour) == n) {
        if (Best_tour(curr_tour))
            Update_best_tour(curr_tour);
    } else {
        for (nbr = n-1; nbr >= 1; nbr--)
            if (Feasible(curr_tour, nbr)) {
                Add_city(curr_tour, nbr);
                Push_copy(stack, curr_tour, avail);
                Remove_last_city(curr_tour);
            }
    }
    Free_tour(curr_tour);
}
```

答:

5. 判断下列程序是否能够安全通信，并说明原因。若不安全，请给出相应解决方案。

```
comm = MPI_COMM_WORLD;
MPI_Comm_rank(MPI_COMM_WORLD, &rank);

if (rank == 0) {
    MPI_Recv(x2, 3, MPI_INT, 1, tag, comm, &status); // A 语句
    MPI_Send(x1, 3, MPI_INT, 1, tag, comm);           // B 语句
}

if (rank == 1) {
    MPI_Recv(x1, 3, MPI_INT, 0, tag, comm, &status); // C 语句
    MPI_Send(x2, 3, MPI_INT, 0, tag, comm);           // D 语句
}
```

答:

三、程序阅读题

1. 补全如下代码片段，以实现 MPI 程序计时并报告运行时间。

```
double local_start, local_finish, local_elapsed, elapsed;
...
_____ ;
local_start = _____ ;

/* Code to be timed */

local_finish = _____ ;
local_elapsed = local_finish - local_start;
_____ ;

if (my_rank == 0)
    printf("Elapsed time = %e seconds\n", elapsed);
```

答:

2. 写出下述程序的运行结果。

```
int k = 100;
#pragma omp parallel for private(k)
for (i = 0; i < 2; i++) {
    k += i;
    printf("k=%d\n", k);
}
printf("last k=%d\n", k);
```

答:

3. 写出以下程序段的运行结果；若删除 `#pragma omp parallel num_threads(2){}`，程序段运行结果又如何？

```
int j = 0;

#pragma omp parallel num_threads(2)
{
    #pragma omp for
```

```
    for (j = 0; j < 2; j++)  
        printf("j=%d, ThreadId=%d\n", j, omp_get_thread_num());  
}
```

答:

四、程序题

1. 请将以下程序改写为 10 个线程执行的 OpenMP 并程序，以使用莱布尼茨级数估计 pi。

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char* argv[]) {
    long long n, i;
    double factor;
    double sum = 0.0;
    n = 1000;

    for (i = 0; i < n; i++) {
        sum += factor / (2*i + 1);
        factor = -factor;
    }

    sum = 4.0 * sum;
    printf("Our estimate of pi = %.14f\n", sum);
    return 0;
}
```

答:

2. 采用 MPI 实现梯形积分。要求：(1) 主函数采用点对点通信，即 0 号进程接收消息并打印，其它进程发送消息；(2) 接收用户输入函数 Get_input 采用 MPI_Bcast 实现。已给 Trap 函数如下。

```
double Trap(
    double left_endpt /* in */,
    double right_endpt /* in */,
    int trap_count /* in */,
    double base_len /* in */) {
    double estimate, x;
    int i;
    estimate = (f(left_endpt) + f(right_endpt)) / 2.0;
```

答：

[illegible]

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#include "queue_1k.h"

const int MAX_MSG = 10000;
void Usage(char* prog_name);
void Send_msg(struct queue_s* msg_queues[], int my_rank,
              int thread_count, int msg_number);
void Try_receive(struct queue_s* q_p, int my_rank);
int Done(struct queue_s* q_p, int done_sending, int thread_count);

int main(int argc, char* argv[]) {
    int thread_count;
    int send_max;
    struct queue_s** msg_queues;
    int done_sending = 0;

    if (argc != 3) Usage(argv[0]);
```



```

thread_count = strtol(argv[1], NULL, 10);
send_max = strtol(argv[2], NULL, 10);
if (thread_count <= 0 || send_max < 0) Usage(argv[0]);

msg_queues = malloc(thread_count * sizeof(struct queue_node_s*));

#pragma omp parallel num_threads(thread_count) \
    default(none) shared(thread_count, send_max, msg_queues, done_sending)
{
    int my_rank = omp_get_thread_num();
    int msg_number;
    srandom(my_rank);
    msg_queues[my_rank] = Allocate_queue();

    #pragma omp barrier
    for (msg_number = 0; msg_number < send_max; msg_number++) {
        Send_msg(msg_queues, my_rank, thread_count, msg_number);
        Try_receive(msg_queues[my_rank], my_rank);
    }

    #pragma omp atomic
    done_sending++;

    while (!Done(msg_queues[my_rank], done_sending, thread_count))
        Try_receive(msg_queues[my_rank], my_rank);

    Free_queue(msg_queues[my_rank]);
    free(msg_queues[my_rank]);
}
free(msg_queues);
return 0;
}

int Done(struct queue_s* q_p, int done_sending, int thread_count) {
    int queue_size = q_p->enqueued - q_p->dequeued;
    if (queue_size == 0 && done_sending == thread_count)
        return 1;
    else
        return 0;
}

```

答:
